

Ports Of Call '26

Enterprise Applikationen

Technische Dokumentation

Fachhochschule Vorarlberg
Software and Information Engineering

Betreut von
Christoph Loacker, Peter Stadelwieser

Vorgelegt von
Esma Canbaz, Irsad Celik, Ali Kilci, Hüdaverdi Mutlu

Dornbirn, 30.06.2026

Inhalt

1. Overview.....	3
1.1 Einleitung	3
Domäne in einem Satz	3
1.2 User Story Map	3
1.3 MVP	5
1.4 UI-Mockups.....	6
.....	8
1.5 Scrum-Regeln (Definition of Ready / Definition of Done)	9
Definition of Ready (DoR) — wann darf eine Story in den Sprint?	9
Definition of Done (DoD) — wann gilt eine Story als fertig?	9
1.6 Event Storming	10
2. Architektur	12
Kontextdiagramm (Ebene 1)	12
Komponentendiagramm (Ebene 2) — Microservices & Infrastruktur	13
Begründung der Abbildung.....	14
2.1 Project Design — Bausteinsicht.....	15
Verantwortlichkeiten der Bausteine	15
API-Gateway als einziger Eingang	16
Frontend-Bausteine	17
2.2 Conceptual Design — Domäne, Laufzeit, Persistenz.....	18
Domänenmodell (vereinfacht)	18
Laufzeitsicht: der Tick-Zyklus	19
Persistenz.....	20
Kommunikationsmuster	20
Ausgewählte Spielmechaniken (mit konkreten Werten)	21
Bank, Kredite & Hypotheken (service-player).....	23
3. Technical Aspects	25
3.1 Tech-Stack.....	25
3.2 Architecture Decision Records (ADR).....	26
ADR-001 · Programmiersprache Backend: Java 21	26
ADR-002 · Frontend: JavaScript mit React 19 + Vite	26
ADR-003 · Datenbank: PostgreSQL.....	26
ADR-004 · Architektur-Einstieg: Monolith zuerst	27
ADR-005 · Schnitt in fünf Microservices + Gateway	27

ADR-006 · Server-autoritatives Spielmodell mit Tick-Engine	27
ADR-007 · Echtzeit-Kommunikation über WebSocket (STOMP)	28
ADR-008 · Synchrone Service-Kommunikation via RestTemplate	28
ADR-009 · Database-per-Service in einem PostgreSQL-Container	28
ADR-010 · Keine gemeinsame „Common Library“ für DTOs.....	28
ADR-011 · Domänenmodell nur dort, wo es Wert schafft.....	29
ADR-012 · Schiffbesicherte Hypothek als Finanz-Hebel	29
ADR-013 · API-Gateway als einziger Eingang (Spring Cloud Gateway)	29
ADR-014 · Optimistic Locking gegen konkurrierende Auftragsannahme.....	30
ADR-015 · Distanzbasierte Schiffsgeschwindigkeit & Spritverbrauch pro Klasse	30
ADR-016 · Containerisierung & Deployment via Docker Compose	31
ADR-017 · Keine Authentifizierung/Autorisierung im MVP	32
4. Nicht-funktionale Anforderungen	33
4.1 Qualitätsziele (priorisiert)	33
4.2 Umsetzung je Qualitätsziel.....	34
Q1 — Konsistenz & Integrität (server-authoritative)	34
Q2 — Echtzeit & Performance	34
Q3 — Skalierbarkeit	34
Q4 — Robustheit & Verfügbarkeit	34
Q5 — Wartbarkeit & Modifizierbarkeit	35
Q6 — Betreibbarkeit & Portabilität.....	35
4.3 Bewusste Nicht-Ziele	35
5. Glossar	36

1. Overview

1.1 Einleitung

Ports of Call ist eine webbasierte, verteilte **Wirtschaftssimulation** für mehrere Spieler. Konzeptionell lehnt sie sich an das klassische gleichnamige Spiel an, baut es aber bewusst **nicht 1:1 nach**. Im Vordergrund steht nicht die grafische Qualität, sondern eine **komplexe, zustandsbehaftete Domäne** aus Handel, Logistik, Risiko und Wettbewerb.

Jeder Spieler leitet eine eigene **Reederei**, kauft und betreibt **Frachtschiffe**, nimmt **Frachtaufträge** zwischen Häfen an, plant **Routen** und trifft strategische Entscheidungen unter Unsicherheit. Mehrere Spieler (2–6 pro Session, MVP-Ziel 4) konkurrieren in **derselben Spielwelt** und in **Echtzeit** um dieselben Markt-Chancen.

Das wichtigste Architekturprinzip ist **serverzentriert** (*server-authoritative*): Sämtlicher relevanter Zustand wird serverseitig gehalten, alle Entscheidungen werden serverseitig berechnet und validiert, und alle Zufallsereignisse werden serverseitig ausgewürfelt. Der Client ist für **Darstellung** und für die **Minispiele** zuständig, die als Reaktion auf Server-Ereignisse erscheinen.

Zeitmodell. Die Spielzeit ist in **Ticks** getaktet: *1 Tick = 1 Spieltag*. Ein zentraler Scheduler im Backend treibt die Zeit voran (Standard alle **5 Sekunden**) und stößt damit Schiffsbewegungen, Markt-Aktualisierungen und Zufallsereignisse an. Über **WebSockets** werden alle Clients einer Session synchron gehalten.

Domäne in einem Satz

Mehrere Reedereien transportieren Fracht zwischen Häfen, verdienen (oder verlieren) dabei Geld und versuchen, am Ende der Session an der Spitze des Leaderboards zu stehen.

1.2 User Story Map

Vor dem Schnitt des MVP wurde der gesamte angedachte Funktionsumfang als User Story Map aufgespannt. Die Backbone (die gelben Spalten oben) gliedert das Spiel in neun fachliche Aktivitäten; darunter hängen die einzelnen Stories, priorisiert nach MoSCoW. Aus dieser Priorisierung wurde unmittelbar der MVP-Umfang (Abschnitt 1.3) abgeleitet: Die MUST-Stories bilden das spielbare Grundgerüst, SHOULD und COULD sind Erweiterungen.

Legende (MoSCoW):

Markierung	Bedeutung	Farbe im Board
MUST	zwingend für das MVP	orange
SHOULD	wichtig, aber nicht MVP-kritisch	rosa

COULD

optionales „Nice-to-have“

grün

Backbone: Game loop · Bank · Company · Ship market · Freight market · Harbour · Session · Random event · Ship

Die folgende Abbildung zeigt die vollständige Story Map mit allen Stories je Backbone-Aktivität, farblich nach MoSCoW priorisiert.



Bezug zum MVP: Alle MUST-Stories sind umgesetzt. Die Bank-Stories (im Board nur als COULD markiert) wurden sogar über das MVP hinaus realisiert — inklusive Krediten und schiffbesicherten Hypotheken. Auch das „global newspaper“ (im Board als COULD priorisiert) wurde inzwischen umgesetzt. Bewusst offen blieb vor allem persist progress (Save/Resume) (siehe Abschnitt 1.3).

1.3 MVP

Das MVP ist ein **spielbares Multiplayer-System ohne KI-Gegner**. Die Spielerzahl pro Session ist konfigurierbar (**2–6**, Auswahl im „New Session“-Formular); das MVP-Ziel ist eine Runde mit **bis zu 4 Spielern**. Die folgende Tabelle zeigt den **Ist-Stand** gegenüber den geforderten MVP-Funktionen.

MVP-Bereich	Geforderte Funktion	Status	Umsetzung im Code
Reedereiverwaltung	Kontostand, Flotte, Leaderboard	✓	BankController#getOverview, FleetView, GET /api/players/session/{id}/leaderboard (von service-player bereitgestellt; service-session ruft ihn beim Endgame ab)
Frachtmarkt	Aufträge generieren	✓	CargoContractRefillService (6 offene Aufträge pro Hafen)
	Annahme im Herkunftshafen	✓	POST /api/cargo/accept
	Abrechnung im Zielhafen	✓	closeOnArrival → Reward-Event per WebSocket
	Regresskontrollen	✓	Forfeit-Strafe (50 %) + Deadline-Penalty (80/Tick)
Schiffsmarkt	Schiffe kaufen	✓	POST /api/ships/buy, 12 Markt-Schiffe (6 neu, 6 gebraucht)
	Schiffe verkaufen	✓	POST /api/ships/{shipId}/sell (Verkaufspreis via SellPriceCalculator)
Schiffssystem	Verwaltung, Fahrten, Einstellungen	✓	Geschwindigkeit/Tanken/Route vor dem Auslaufen
Häfen	Unterschiedliche Eigenschaften	✓	Seed aus navigation-data.json, Zoll- & Lotsen-Logik
	Reparatur & Betankung	✓	POST /api/ships/repair, /api/ships/refuel
Ereignisse	Lotsenstreik / manuelles Anlegen	✓	ParkingGame (manuelles Anlegen als Minispiel)
	Schmuggel + Zoll + Bestechung	✓	SmugglingController, CustomsController
Session	Erstellen / Beitreten / Beenden	✓	SessionController, End-Game-Vote

Über das MVP hinaus bereits umgesetzt:

- **Bank** mit Krediten (Loan): Aufnahme, automatische Tilgung pro Tick, Status ACTIVE → OVERDUE → DEFAULTED, Bonitäts-Rating (S+ bis E).
- **Hypothecken** (Mortgage): durch ein Schiff **besicherte** Kredite bis 60 % des Schiffswerts; bei zu vielen verpassten Raten **pfändet** die Bank das Schiff (SEIZED). Siehe Abschnitt „Bank, Kredite & Hypothecken“.
- **Politisches Weltereignis**: Routen-Sperren (Suezkanal, Panamakanal) mit automatischer **Alternativroute**; Hafen-Sperren (Nordatlantik, Straße von Hormuz).
- **Zufallsereignisse als Minispiele**: *Ratten an Bord* (Cargo-/Kabel-Variante), *Mann über Bord*, *Felsen/Klippen ausweichen* (Steuer-Minispiel) und *manuelles Anlegen*.
- **Flüssige Schiffsanimation** auf der Weltkarte (zeitbasierte 60-FPS-Interpolation).
- **Globale Spielzeitung** (Newspaper): session-weite News-Einträge — etwa anonymisierte Schmuggel-Busts und politische Ereignisse — verwaltet von service-session (*NewsController*, *NewsItem*, *NewsService*) und im Frontend als *NewspaperView* dargestellt.

Offene Punkte / bewusst nicht umgesetzt: weitere Zufallsereignisse (Eisberg, Sturmtief), Save/Resume nach Disconnect. Diese sind im MVP optional bzw. wurden zugunsten von Stabilität und Präsentationsreife zurückgestellt.

1.4 UI-Mockups

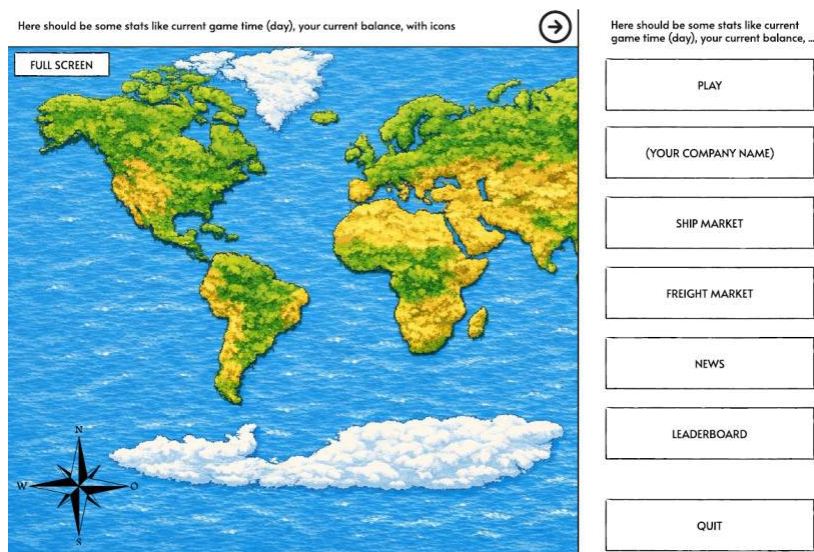
Die **UI-Mockups** wurden in **Figma** erstellt. Sie sind bewusst **schlicht** gehalten — überwiegend **grau und ohne Farbe**, mit Wireframe-Charakter — und legen vor allem das Layout und die Informationsarchitektur fest: eine zentrale Weltkarte mit seitlichen Menüs für Hafen, Broker, Büro und Bank sowie Platzhalter-Tabellen für Märkte, Verträge, Leaderboard und Ereignisse. Ergänzend liegen Schiffs-/Karten-Vorlagen als Bilddateien im Projektordner (1.png ... 9.png, worldmap.png).

Das heutige **UI übernimmt** diese **Grundstruktur**, geht **gestalterisch und technisch** aber deutlich **darüber hinaus**:

- **Durchgängiger Pixel-Art-Look**: Nicht nur die Weltkarte, sondern alle Screens — Login/Lobby, Ship Broker, Hafenansicht — sind durchgehend im Pixel-Art-Stil gehalten und teils animiert (z. B. zeitbasierte 60-FPS-Schiffsbewegung auf der Karte, Canvas-Hintergrund im Broker). Wo die Mockups grau und schlicht sind, ist das fertige UI farbig ausgearbeitet.
- **Einheitliches Farb- und Typografiesystem**: dunkelblaue Flächen mit goldgelben Rahmen und Akzenten, ein blauer Interaktions-Akzent und eine klare UI-Hierarchie sorgen für ein konsistentes, maritimes Erscheinungsbild.

- **Single-Screen-Dashboard** statt Multi-Page-App: Die Weltkarte ist dauerhaft als Hintergrund (100vh) gerendert; die **interaktiven Menüs** (Harbour, Broker, Office, Bank) gleiten als leicht transluzente **Overlay-Sidebar** rechts ein.
- **Ereignisse und Minispiele direkt eingebettet: Weltereignisse** (z. B. „Panama Canal closed“) erscheinen als **Banner/Modal**, **Minispiele** (manuelles Anlegen, Hindernis-Navigation, Ratten, Mann über Bord) als **Vollbild-Overlays** über der weiterlaufenden Karte — nicht als separate Seiten.

Die folgenden Screenshots zeigen den aktuellen Stand - im Vergleich zu den schlichten Figma-Mockups lässt sich die Entwicklung des UI gut nachvollziehen.



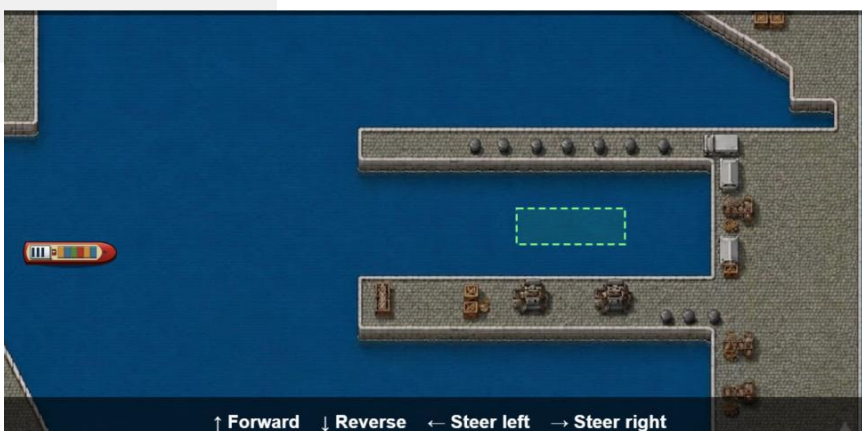


MANUAL DOCKING

STEER YOUR SHIP INTO THE MARKED DOCK WITHOUT HITTING THE WALLS

HULL :	85%	TIME :	0:42	PRECISION : 84 %
PRECISION : 84 %				
QUAY WALL				
MY SHIP >>>				
DOCK HERE				
QUAY WALL				
W A S D / Arrow Keys - Steer SHIFT - Boost ESC - Abort				

ABORT (PAY FEE)



HAMBURG

REGION : Northern Europe | COUNTRY : Germany

PORT LOCATION ON WORLD MAP:

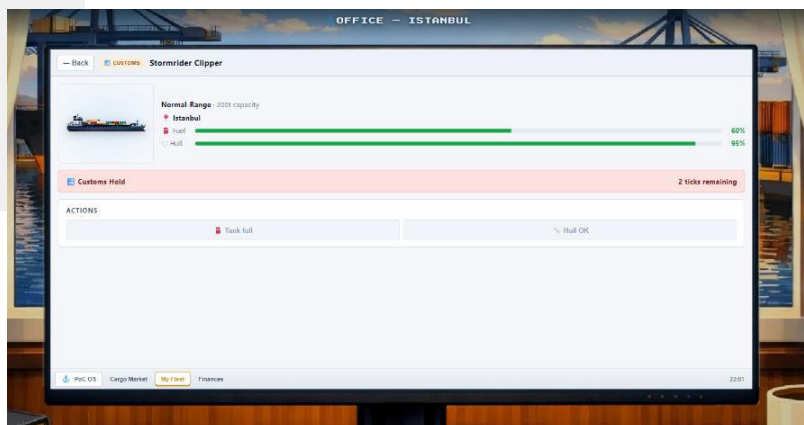
PORT INFO		RISK LEVEL	
HOME PORT COST :	XXX USD	CUSTOMS INTENSITY :	MEDIUM
PILOT FEE :	XXX USD	PILOT STRIKE RISK :	LOW
REPAIR COST :	XXX / unit		
REFUEL COST :	XXX / unit		
ECONOMIC ATTRACTIVENESS		AVAILABLE GOODS	
ATTRACTIVENESS :	HIGH	STEEL COAL GRAIN TIMBER	
ACTIVE CONTRACTS :	XXX	ELECTRONICS	
DEMAND PROFILE :	Industrial		

BACK

REPAIR SHIP

REFUEL SHIP

SET AS HOME PORT



1.5 Scrum-Regeln (Definition of Ready / Definition of Done)

Das Projekt wurde nach Scrum organisiert. Die folgenden Kriterien sind die Scrum-Regeln des Teams.

Definition of Ready (DoR) — wann darf eine Story in den Sprint?

„Ready“ steht für ein gemeinsames Verständnis zwischen Product Owner und Entwicklungsteam darüber, welche Informationen und Vorbereitung vorliegen müssen, bevor die eigentliche Entwicklung beginnen kann.

- Es gibt **Akzeptanzkriterien** — definiert und **testbar**; sie legen fest, wann eine User Story als angenommen gilt.
- **Alle Abhängigkeiten** zu externen Systemen, Rollen oder Schnittstellen sind geklärt.
- Die Story wurde vom Team **geschätzt**, und das Team **versteht**, wie das Feature in die Produktvision passt.
- Der Product Owner stellt sicher, dass **alle nötigen Informationen** vor Sprint-Start vorliegen; das Team **committet** sich, nur Stories zu ziehen, die diese Kriterien erfüllen.
- Die Story enthält eine klare **Definition von Rollen und Verantwortlichkeiten**, und das **Ziel ist klar definiert und innerhalb eines Sprints erreichbar**.
- Jeder Task hat einen **eigenen Branch**; Branches beginnen stets mit `Feature[Task_name]`.

Definition of Done (DoD) — wann gilt eine Story als fertig?

Ein Backlog-Item gilt als *done*, wenn:

- **alle Akzeptanzkriterien** vollständig erfüllt sind,
- die beschriebenen Funktionen **erfolgreich getestet** wurden,
- das Produkt **ohne kritische Fehler** in der Umgebung läuft,
- der Code **implementiert, reviewed und** mit dem Hauptsystem **gemergt** ist,
- **keine TODOs** und keine temporären Fixes mehr im Code verbleiben,
- die **Dokumentation aktuell** ist und der **Product Owner** das Ergebnis abgenommen hat.

Task-Dokumentation: vollständig im Task beschrieben, mit allen nötigen Implementierungen/Änderungen. **Story-Dokumentation:** vollständig dokumentiert (User Story, Akzeptanzkriterien + Mockup).

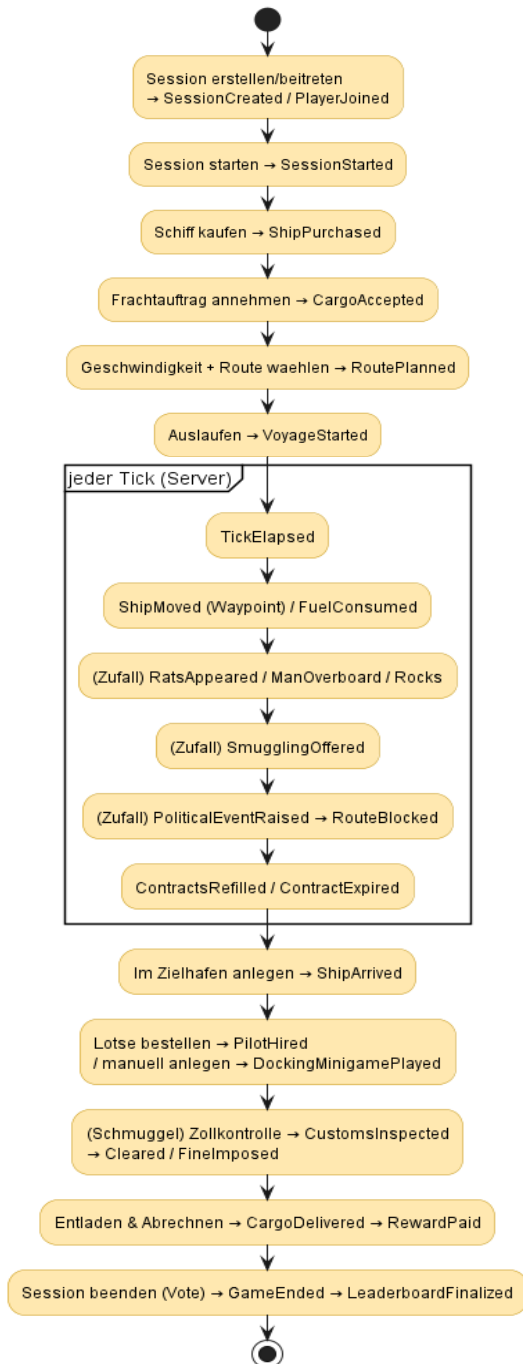
Ergänzend gilt: Code auf **Englisch**, Konfigurationswerte in **Config-Files** statt hartcodiert, und KI-generierter Code muss **verstanden und erklärbar** sein.

1.6 Event Storming

Das Event Storming diente dazu, die Domäne über **Domain Events** zu erschließen und daraus **Commands**, **Aktoren** und die späteren **Bounded Contexts** abzuleiten.



Der zentrale Ablauf einer Reise sieht im Ereignisfluss so aus:



Wichtigste Erkenntnis aus dem Event Storming: Die meisten dieser Events sind **zeit- bzw. servergetrieben**, nicht Request/Response. Daraus folgte direkt die Entscheidung für ein **Tick-Modell mit WebSocket-Broadcast** statt reinem Polling — und die Aufteilung der Event-Cluster in die fünf fachlichen Services (siehe Abschnitt 2).

2. Architektur

Kontextdiagramm (Ebene 1)

Das System hat einen einzigen externen Aktor — den **Spieler im Browser** — und einen einzigen öffentlichen Eingang, das **API-Gateway**.

Es gibt **keine externen Fremdsysteme** (keine Zahlungsanbieter, kein Auth-Provider o. Ä.). Das vereinfacht den Kontext bewusst: Der gesamte Spielzustand lebt innerhalb der Systemgrenze.

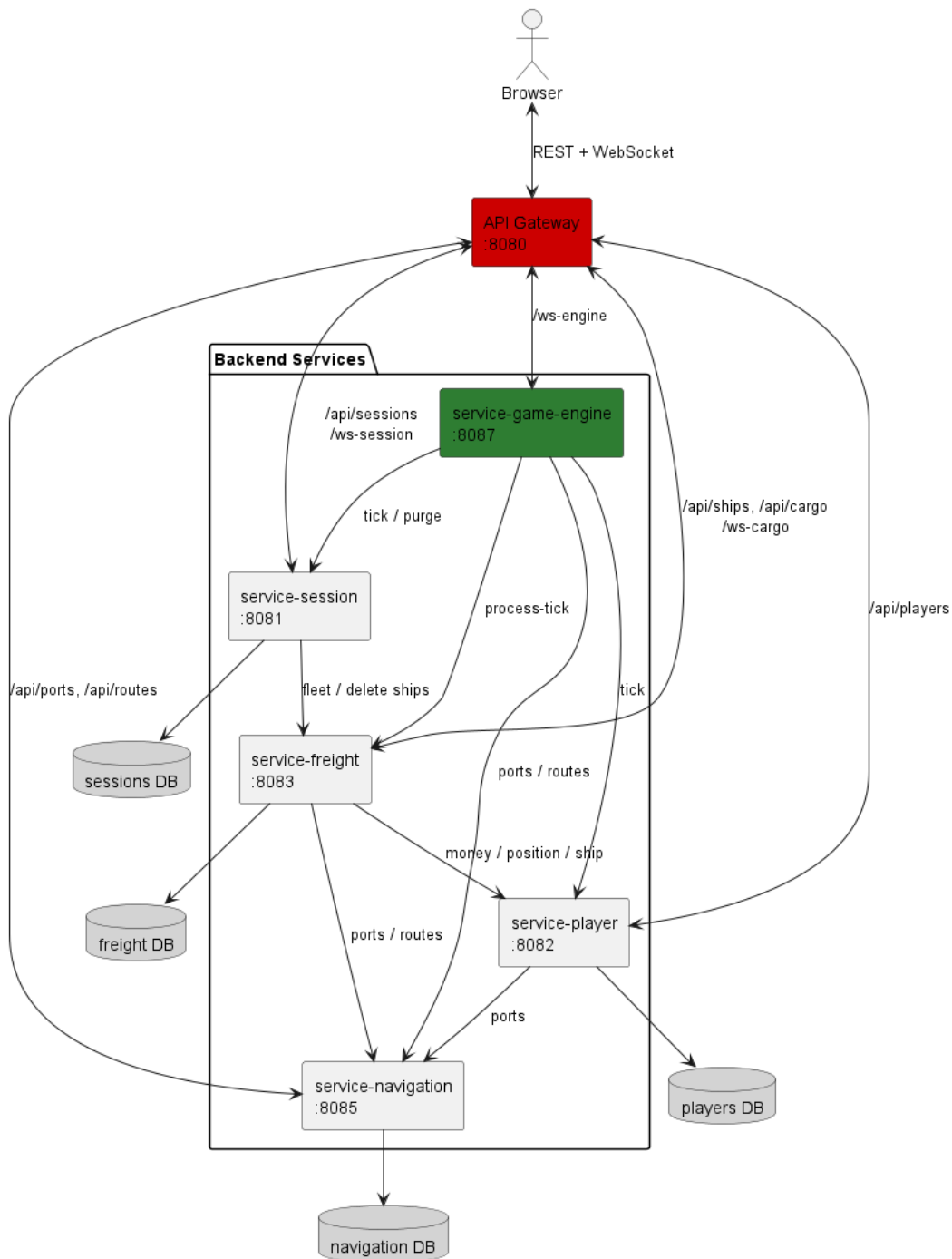
Kontextdiagramm (Ebene 1)



Komponentendiagramm (Ebene 2) — Microservices & Infrastruktur

Die Architektur ist **microservice-geschnitten**: fünf fachlich unabhängige Spring-Boot-Services plus ein API-Gateway. Die Kommunikation läuft synchron über **REST (RestTemplate)** und asynchron/echtzeit über **WebSocket (STOMP)**.

Hinweis: Alle WS-Verbindungen laufen **über das API-Gateway** (/ws-session, /ws-cargo, /ws-engine) und nicht direkt zu den Services.



Begründung der Abbildung

Das Diagramm bildet vier Entwurfsentscheidungen unmittelbar ab; jede ist in einem ADR hinterlegt:

Ein Gateway als einziger Eingang (rot). Der Browser kennt nur eine Adresse (:8080). Das Gateway bündelt Routing, CORS und den WebSocket-Tunnel an genau einer Stelle und ersetzt die @CrossOrigin-Annotationen, die in der Monolith-Zeit an jedem Controller hingen. Dadurch bleibt die öffentliche Angriffsfläche minimal und die internen Service-Ports (8081–8087) sind von außen nicht erreichbar (ADR-005, ADR-013).

Fünf fachlich geschnittene Services statt eines Monolithen. Die Event-Cluster aus dem Event Storming (Abschnitt 1.6) wurden 1:1 zu Bounded Contexts: Session-Lebenszyklus (session), Reederei & Finanzen (player), Schiffsbetrieb/Reise/Zoll (freight), statische Kartendaten (navigation) und Zeitsteuerung (game-engine). Jeder Service besitzt seine Daten exklusiv (eigene DB, grau dargestellt) — es gibt keinen datenbankübergreifenden Zugriff und keine Fremdschlüssel über Servicegrenzen; Referenzen sind reine ID-Strings (ADR-005, ADR-009).

Azyklische Abhängigkeiten. Die Pfeile zeigen bewusst in eine Richtung: navigation liegt ganz unten (statische Stammdaten, keine eigenen Abhängigkeiten), die game-engine ganz oben als reiner Orchestrator. Pro Tick ruft die game-engine die Services in fester Reihenfolge an (tick / process-tick / purge); Die fachlichen Service-zu-Service-Aufrufe im Tick (session→freight, freight→player, freight→navigation, player→navigation) bilden keinen Zyklus. Einzige Ausnahme außerhalb des Tick-Pfads ist die fire-and-forget-News-Meldung freight→session (Schmuggel-/Vertrags-News); zusammen mit session→freight entsteht hier formal ein Zyklus. Da die Meldung fire-and-forget ist (Fehler brechen die Cargo-/Customs-Verarbeitung nicht ab) und nicht im Orchestrierungspfad der game-engine liegt, bleibt der Tick selbst azyklisch. Das hält die Services unabhängig deploybar und verhindert gegenseitige Blockaden im Tick.

Die game-engine ist separat und stateless. Die Zeitsteuerung ist ein Querschnitts-Concern, der alle Services betrifft — sie gehört in keinen der fachlichen Kontexte. Als eigener, zustandsloser Dienst (grün, ohne eigene DB) koordiniert sie nur und hält selbst keinen Spielzustand; stürzt sie ab und startet neu, geht nichts verloren, weil der Zustand in den anderen Services liegt (ADR-006).

Zwei Kommunikationswege erklären die unterschiedlichen Pfeiltypen:

- **WebSocket über das Gateway.** Alle drei WS-Verbindungen (/ws-session, /ws-cargo, /ws-engine) laufen über das Gateway statt direkt zu den Services. So spricht der Browser auch für Echtzeit nur einen Origin an; das Gateway reicht den HTTP-Upgrade durch und tunnelt die STOMP-Frames an den jeweiligen Upstream (ADR-007, ADR-013).
- **REST + WebSocket gemischt.** Punktuelle Daten zwischen Services (Reward-Buchung, Port-/Routen-Lookup, Tick-Anstoß) laufen synchron über RestTemplate — einfach und ohne zusätzlichen Message-Broker. Der zeitgetriebene Massen-Push (Tick, GameState) läuft asynchron über WebSocket, weil reines Request/Response für das Tick-Modell unpassend wäre (ADR-007, ADR-008).

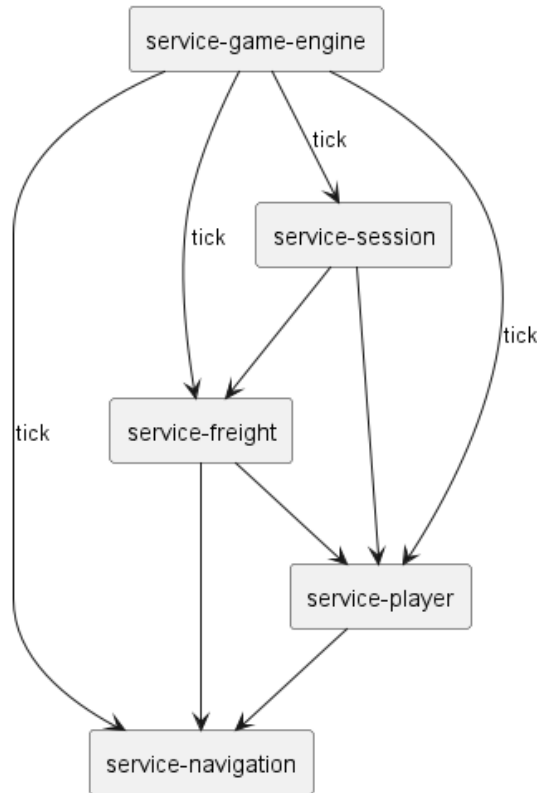
2.1 Project Design — Bausteinsicht

Verantwortlichkeiten der Bausteine

Service	Verantwortung	Owned Entities	Hängt ab von	Port
api-gateway	Einzigster öffentlicher Eingang; Routing & CORS für REST + WS	—	alle	8080
service-session	Spielsitzungen: Lobby, Start, Tick-Zähler, End-Game-Vote, Session-Code	GameSession, SessionPlayer, NewsItem	player, freight	8081
service-player	Spielerprofil, Kontostand, Position, Heimathafen, Bank/Kredite & Hypotheken	Player, Loan, LoanTransaction, Mortgage, MortgageTransaction	navigation, freight	8082
service-freight	Schiffskauf/-betrieb, Routenausführung, Frachtverträge, Zoll & Schmuggel	PlayerShip, MarketShip, CargoContract	player, navigation, session (nur fire-and-forget-News)	8083
service-navigation	Statische Kartendaten: Häfen, Routen, Wegpunkte, politische Ereignisse	Port, Route	—	8085
service-game-engine	Tick-Orchestrierung, Schiffsbewegung, WS-Broadcast (stateless)	—	alle	8087

Die Abhängigkeiten zeigen **in eine Richtung** (azyklisch): navigation ganz unten ohne eigene Abhängigkeiten, game-engine ganz oben als Orchestrator.

Microservice-Kommunikation (Service-Abhängigkeiten)



API-Gateway als einziger Eingang

Das Gateway (Spring Cloud Gateway, reaktiv/WebFlux) bündelt:

- **Routing** per Pfad-Prädikat: `/api/sessions/**` → session, `/api/ships/**`, `/api/cargo/**`, `/api/customs/**` → freight, `/api/players/**`, `/api/bank/**`, `/api/loans/**`, `/api/mortgages/**` → player, `/api/ports/**`, `/api/routes/**`, `/api/political-events/**` → navigation.
- **WebSocket-Tunnel**: Das `ws://`-Schema im Routing-Eintrag weist das Gateway an, den HTTP-Upgrade durchzureichen und STOMP-Frames an den Upstream zu tunneln.
- **Globales CORS** — ersetzt die früheren `@CrossOrigin`-Annotationen pro Controller aus der Monolith-Zeit.

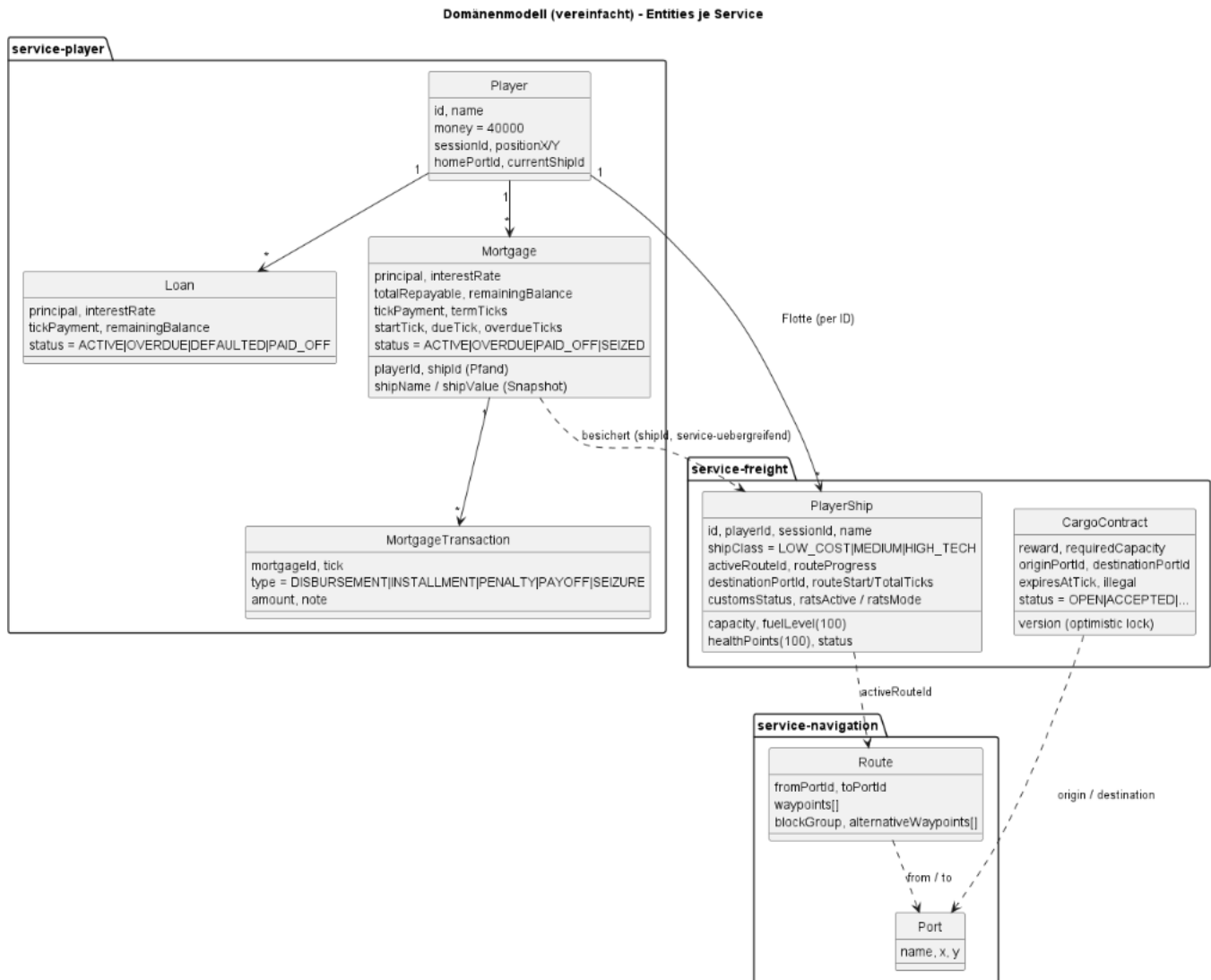
Frontend-Bausteine

Das Frontend ist eine **React-19-SPA** (Vite-Build) **ohne Router** — der Zustand `currentView` im `GameContext` steuert, welche Overlay-Ansicht erscheint.

Baustein	Aufgabe
<code>GameContext</code>	Zentraler State + STOMP-Client (<code>@stomp/stompjs</code>): abonniert <code>/topic/game/{id}</code> , <code>/topic/lobby/{id}</code> , <code>/topic/customs/{id}</code>
<code>ToastContext</code>	Globale, nicht-blockierende Hinweis-Meldungen (ersetzt <code>Inline-Error-Banner</code>)
<code>Worldmap</code>	Dauerhafte Hintergrundkarte; zeitbasierte 60-FPS-Interpolation der Schiffe
<code>HarbourView</code> , <code>BrokerView</code> , <code>CargoMarketView</code> , <code>FleetView</code> , <code>BankView</code> , <code>OfficeView</code>	Overlay-Ansichten der Sidebar
<code>ParkingGame</code> , <code>RatsMinigameView</code> , <code>ManOverboardView</code> , <code>MinigameView</code> (Felsen-Steuern)	Minispiele (Reaktion auf Server-Events)
<code>BankView</code> (Kredit- & Hypotheken-Sicht)	Aufnahme/Tilgung von Krediten und schiffbesicherten Hypotheken
<code>CustomsModal</code> , <code>SmugglingOfferModal</code> , <code>PilotModal</code> , <code>EndGameModal</code>	Ereignis-Dialoge
<code>MapEditor</code>	In-Browser-Werkzeug, mit dem <code>navigation- data.json</code> (Häfen + Wegpunkte) erzeugt wird

2.2 Conceptual Design — Domäne, Laufzeit, Persistenz

Domänenmodell (vereinfacht)



Geplantes vs. umgesetztes Modell. Das ursprünglich entworfene Domänenmodell (mit Klassen wie Harbor, FreightOffer, FreightContract, ShipOffer, RandomEvent, Bank und Attributen wie balance/ownerId) wich an einigen Stellen vom heutigen Stand ab. Die hier gezeigten Strukturen spiegeln den **tatsächlich implementierten** Stand wider — etwa `Player.money` statt `balance`, ID-Referenzen statt Objekt-Beziehungen, und Frachtangebot + Vertrag zusammengefasst in `CargoContract`.

DDD-Bezug. Nicht alle Services sind gleich „schwer“ modelliert: service-navigation ist im Kern **CRUD** (statische Stammdaten) und kommt ohne reiches Domänenmodell aus, während service-freight die eigentliche **Geschäftslogik** (Reiseausführung, Zoll, Schmuggel) kapselt. Jeder Service besitzt seine Entities exklusiv; **es gibt keine Fremdschlüssel über Service-Grenzen** — Referenzen sind reine ID-Strings (z. B. playerId, destinationPortId). Das hält die Bounded Contexts entkoppelt.

Laufzeitsicht: der Tick-Zyklus

Der TickEngineServiceImpl in der game-engine ist das Herzstück. Er läuft per `@Scheduled(fixedDelayString = "${game.tick.interval-ms:5000})` — **fixedDelay** (nicht fixedRate) verhindert, dass sich Ticks stauen, falls die Verarbeitung einmal länger dauert als das Intervall.

Pro Tick passiert (vereinfacht):

1. session-service → aktive Sessions holen; leere Sessions auto-purgen
2. navigation-service → politische Ereignisse EINMAL global weiterzählen
→ resultierende Events an alle Sessions broadcasten für jede aktive Session:
3. session-service → Tick-Zähler +1 (newTick)
4. freight-service → Frachtaufträge nachfüllen / abgelaufene entfernen
- 5a. player-service → fällige Kreditraten abbuchen (Loan)
- 5b. player-service → fällige Hypothekenraten abbuchen (Mortgage);
bei Zahlungsausfall ggf. Schiff pfänden (SEIZED)
6. freight-service → alle fahrenden Schiffe bewegen
↳ liefert: Ankünfte, Wegpunkt-Updates, Lotsen-Anfragen, Minispiel-Events
7. Seiteneffekte anwenden:
 - Ankunft → player-Position setzen, Frachtvertrag abrechnen
 - Wegpunkt → player-Position aktualisieren
 - Lotse/Minispiel → an betroffenen Spieler broadcasten
8. GameState ZUERST bauen, DANN broadcasten:
 - broadcast Tick-Nachricht → /topic/game/{id}
 - broadcast GameState → /topic/game/{id}

Der GameState wird **vor** dem Broadcast gebaut: Schlägt der Aufbau fehl, werden **beide** Broadcasts übersprungen, statt den Tick-Zähler weiterlaufen zu lassen, während die Schiffe „einfrieren“. So bleiben Tick-Zähler und Schiff-Snapshot im Gleichschritt.

Bewegung. Eine Route ist eine Polylinie aus Wegpunkten; pro Tick rückt das Schiff um eine reale Distanz (`routeProgress += distancePerTick`) entlang der Strecke vor. Das Frontend interpoliert zwischen zwei Broadcasts über `routeProgress + tickFraction × distancePerTick` und misst das Tick-Intervall dynamisch.

Persistenz

- **Vier eigenständige PostgreSQL-Datenbanken** in **einem** Container (postgres:15), per Init-Skript (01-create-databases.sql) angelegt: portsofcall_sessions, portsofcall_players, portsofcall_freight, portsofcall_navigation. Im Diagramm wird die „große Datenbank-Box“ mit den einzelnen Service-DBs darin dargestellt.
- **Database-per-Service**: Jeder Service spricht ausschließlich seine eigene Datenbank an (spring.jpa.hibernate.ddl-auto=update); es gibt keinen datenbankübergreifenden Zugriff.
- service-game-engine ist **stateless** und hat keine eigene Datenbank — sämtlicher Zustand lebt in den anderen Services.
- **Optimistic Locking** auf CargoContract (@Version): Verhindert, dass zwei Spieler denselben Auftrag gleichzeitig annehmen — der zweite Schreibvorgang läuft auf eine OptimisticLockException.

Kommunikationsmuster

Muster	Technik	Einsatz
Synchron (Request/Response)	RestTemplate über HTTP, Service-zu-Service	Leaderboard-Daten, Reward- Buchung, Port-/Routen- Lookup, Tick-Orchestrierung
Asynchron / Echtzeit	WebSocket + STOMP, drei unabhängige Broker (session, freight, engine)	GameState-Broadcast, Lobby- Updates, Zoll-/Schmuggel- /Minispiel-Events

STOMP-Topics: /topic/lobby/{sessionId} (Lobby), /topic/game/{sessionId} (Tick + GameState + Reward/Pilot/Minigame), /topic/customs/{sessionId} (Zollkontrolle).

Ausgewählte Spielmechaniken (mit konkreten Werten)

Die wichtigsten Balance-Parameter liegen als **Konstanten / Config-Werte** vor:

Mechanik	Wert	Wo
Tick-Intervall	5.000 ms (= 1 Spieltag)	game.tick.interval-ms
Startkapital	40.000 Taler (Default, pro Session überschreibbar)	PlayerServiceImpl.DEFAULT_STARTING_MONEY
Betankung	10 Taler / Einheit	REFUEL_COST_PER_UNIT
Reparatur	125 Taler / HP	REPAIR_COST_PER_HP
Mindest-HP zum Auslaufen	20	MIN_HP_TO_SAIL
Lotse bestellen	500 Taler	PILOT_HIRE_COST
Regress bei Vertragsbruch	50 % des Reward	FORFEIT_PENALTY (1/2)
Verspätungsstrafe	80 Taler / Tick	LATE_DEDUCTION_PER_TICK
Zollkontrolle (Wahrscheinlichkeit)	35 %	customs.trigger-probability
Zoll-Festsetzung	1–5 Ticks	customs.min/max-hold-ticks
Zoll-Bearbeitung	1–3 Ticks	customs.min/max-processing-ticks
Bestechungs-Schwelle	0,75 × Schmuggelwert	customs.bribe-threshold-multiplier
Schmuggel-Angebot (Wahrscheinlichkeit)	33 %	smuggling.offer-probability
Politisches Ereignis (Routensperre)	Abstand 8–16 Ticks, Dauer 50–70 Ticks, Karenz 3 Ticks	PoliticalEventServiceImpl
Ratten (Auslöser pro Tick)	10 %	RATS_CHANCE

Ratten-Schaden	2 % / Tick, max. 50 %	RATS_DAMAGE_PER_TICK_PERCENT
Mann über Bord (Wahrscheinlichkeit)	14 %	MAN_OVERBOARD_CHANCE
Mann über Bord: Strafe bei Misserfolg	8–18 HP	OVERBOARD_FAIL_HP_PENALTY_MIN/MAX
Felsen/Klippen (Wahrscheinlichkeit)	14 %	ROCKS_CHANCE
Schiffsverkauf: Wiederverkaufsrate	55 % / 60 % / 65 % (Low-Cost / Medium / High-Tech)	SellPriceCalculator
Schiffsverkauf: Rumpf- & Altersabschlag	Hull-Floor 40 %, Altersabschlag 1 % / h (Floor 80 %)	SellPriceCalculator
Frachtauftrag-Deadline	Basis 6 Ticks, High- Risk-Faktor 0,65	CargoContractRefillService
Lotsenstreik	Abstand 20–40 Ticks, Dauer 35–50 Ticks	PoliticalEventServiceImpl
Min. Kredit / Min. Hypothek	je 1.000 Taler	BankServiceImpl.MIN_LOAN, MortgageServiceImpl.MIN_MORTGAGE
Hypothek: Beleihungsgrenze (LTV)	60 % des Schiffswerts	mortgage.max-ltv
Hypothek: Verzugsstrafe	5 % der Restschuld / Tick	mortgage.overdue.penalty-rate
Hypothek: Pfändung nach	5 Verzugs-Ticks	mortgage.overdue.max-ticks
Kredit: Verzugsstrafe / Ausfall nach	5 % / Tick · 5 Verzugs-Ticks	loan.overdue.penalty-rate / .max-ticks

Schiffsklassen (Seed aus MarketShipDataInitializer, je 6 neue + 6 gebrauchte):

KLASSE	BEISPIEL	PREIS	KAPAZITÄT	VERBRAUCH / 100 DIST.	GESCHW.
LOW_COST_SHIPS	Rusty Sloop	10.000	50	20	55
MEDIUM_COST_SHIPS	Steady Trader	25.000	150	12	42
HIGH_TECH_SHIPS	Ocean Titan	80.000	500	7	30

Gebrauchte Schiffe starten mit **50–70 HP** und sind entsprechend günstiger ($\text{preis} \times (1 - (100 - \text{HP}) \cdot 0,006)$).

Bestechung beim Zoll ist bewusst riskant umgesetzt: Die Erfolgswahrscheinlichkeit ist $\min(1, \text{gezahlterBetrag} / \text{Schwelle})$. Schlägt der Versuch fehl, ist die gezahlte Bestechung verloren und die Entdeckungswahrscheinlichkeit steigt auf 100 % (volle Konfiszierung + Bußgeld)."

Politische Ereignisse folgen einer kleinen Zustandsmaschine Normal → Grace → Enforced → Normal (in der Grace-Phase dürfen Schiffe noch passieren, erst ab *Enforced* greift die Sperre):

- **Routen-Sperren** (SUEZ_CANAL, PANAMA_CANAL): Die betroffenen Routen werden gesperrt. Für Hafenpaare, zu denen ein Detour vorberechnet wurde, weicht das Schiff automatisch auf die vorberechnete Alternativroute aus (per Dijkstra über einen Alternativ-Subgraphen geseedet); Routen ohne hinterlegten Detour bleiben für die Dauer der Sperre vollständig geschlossen.
- **Hafen-Sperren** (NORTH_ATLANTIC, STRAIT_OF_HORMUZ): blockieren bestimmte Häfen ohne Alternative.

Die Routen selbst werden beim Start von *service-navigation* aus navigation-data.json geladen (erzeugt vom In-Browser-MapEditor) und per **Dijkstra-Kürzeste-Pfad** zwischen allen Hafenpaaren bidirektional verstickt.

Bank, Kredite & Hypotheken (service-player)

Die Finanzmechanik liegt vollständig in service-player und besteht aus drei Bausteinen: einer **Bonitätsprüfung** (BankService), **unbesicherten Krediten** (Loan) und **schiffbesicherten Hypotheken** (Mortgage). Beide Kreditarten werden pro Tick zentral von der Game-Engine bedient (POST /api/loans/process-tick, POST /api/mortgages/process-tick).

Bonität. Aus *Cash* und *Gesamtschulden* errechnet der *BankServiceImpl* einen *Credit-Score* (das Reinvermögen wird zusätzlich ausgewiesen):

Rating	Score-Schwelle	Basiszins p. Laufzeit
S+	≥ 850	3 %
A	≥ 700	5 %
B	≥ 500	8 %
C	≥ 300	12 %
D	≥ 150	18 %
E	< 150	25 %

Der Basiszins wird über die Laufzeit angepasst ($\text{base} + (\text{termTicks}/60 - 1) \cdot 0,02$, Untergrenze 2 %), sodass kürzere Laufzeiten günstiger sind. Die Mindestsumme je Kredit/Hypothek beträgt **1.000 Taler**.

Kredit (Loan). Unbesichert; Auszahlung erhöht sofort den Kontostand. Pro Tick wird die fällige Rate (*tickPayment*) abgebucht. Kann der Spieler nicht zahlen, wird der Kredit **OVERDUE**, es fällt **5 % Verzugszins auf die Restschuld pro Tick** an, und nach **5 aufeinanderfolgenden Verzugs-Ticks** fällt er auf **DEFAULTED**. Status: **ACTIVE** → **OVERDUE** → **DEFAULTED** bzw. → **PAID_OFF**.

Hypothek (Mortgage). Mechanisch wie ein Kredit, aber durch ein **Schiff besichert**: Die Beleihungsgrenze beträgt **60 % des Schiffswerts** (*max-ltv*), das Schiff trägt einen sichtbaren **Pfand-Marker** und kann nicht verkauft werden, solange die Hypothek offen ist. Bei Zahlungsausfall greift dieselbe Verzugslogik (5 %/Tick); nach **5 Verzugs-Ticks pfändet** die Bank jedoch das Schiff (*shipClient.seizeShip*, Status **SEIZED**) und schreibt die Restschuld ab — statt eines reinen **DEFAULTED** wie beim Kredit. Status: **ACTIVE** → **OVERDUE** → **SEIZED** bzw. → **PAID_OFF**. Jede Buchung wird als *MortgageTransaction* (**DISBURSEMENT**, **INSTALLMENT**, **PENALTY**, **PAYOFF**, **SEIZURE**) protokolliert, was eine vollständige Hypotheken-Historie ermöglicht.

Designentscheidung: Die Hypothek erweitert die Bank um ein **Risiko-/Hebel-Instrument** — Liquidität gegen den Einsatz eines Schiffes als Sicherheit — und ergänzt damit den unbesicherten Kredit (siehe ADR-012).

3. Technical Aspects

3.1 Tech-Stack

Schicht	Technologie	Version	Begründung (Kurz)
Sprache Backend	Java	21 (LTS)	Team-Erfahrung, starkes Ökosystem für verteilte Systeme
Framework Backend	Spring Boot	4.0.5	Web, Data JPA, WebSocket „aus einem Guss“
Build Backend	Gradle (Multi-Modul)	Toolchain 21	Ein Repo, ein Build für 6 Module
API-Gateway	Spring Cloud Gateway (WebFlux)	—	Routing + CORS + WS-Tunnel an einem Punkt
Persistenz	PostgreSQL	15	Relationale Domäne, JPA/Hibernate-Integration
Sprache Frontend	JavaScript + React	19.2	Komponenten, Team-Erfahrung
Build Frontend	Vite	8	Schneller Dev-Server + Hot Reload
Echtzeit	WebSocket + STOMP (@stomp/stompjs 7.3)	—	Server-Push für Tick-Synchronisation
Deployment	Docker Compose	—	Ein up startet DB + alle Services + Frontend

3.2 Architecture Decision Records (ADR)

Die **ADRs** folgen dem Aufbau **Status · Kontext · Entscheidung · Begründung · Konsequenzen** und sind mit Kürzeln nummeriert. ADR-001 bis ADR-003 bilden die Kern-Records (Programming Language, Database, Microservices); ADR-004 bis ADR-017 dokumentieren die weiteren Architekturentscheidungen, die im Lauf der Umsetzung getroffen wurden.

ADR-001 · Programmiersprache Backend: Java 21

- **Status:** ✓ Akzeptiert
- **Kontext:** Verteilte Multiplayer-Web-Anwendung mit mehreren Backend-Services, synchronisierter Spiellogik und serverseitiger Validierung.
- **Entscheidung:** **Java 21** (aktuelles LTS) als Sprache und Laufzeit für das Backend.
- **Begründung:**
 - Das Team kennt Java bereits aus früheren Lehrveranstaltungen — schnellere Umsetzung.
 - Hervorragende Integration mit **Spring Boot** für Web und Microservices.
 - Starkes Ökosystem für verteilte Systeme, REST, Messaging, Nebenläufigkeit.
 - LTS sichert Wartbarkeit über die Projektlaufzeit hinaus; moderne Sprachfeatures (Records, Pattern Matching, virtuelle Threads).
- **Konsequenzen:** Schnellere Implementierung, gute Wartbarkeit, einfache Integration von Enterprise-Frameworks und Test-Werkzeugen.

ADR-002 · Frontend: JavaScript mit React 19 + Vite

- **Status:** ✓ Akzeptiert
- **Kontext:** Webbasierter Multiplayer-Client mit interaktiven UI-Komponenten (Session-, Markt-, Flotten-Ansichten) und Echtzeit-Updates.
- **Entscheidung:** **React 19** (JavaScript) mit **Vite** als Frontend-Stack.
- **Begründung:**
 - Team-Erfahrung aus früheren Web-Projekten.
 - Komponentenbasierte Architektur → wiederverwendbare UI-Elemente (Hafen-, Schiffsmarkt-, Frachtansichten).
 - Vite: schneller Start + Hot Reload → schnelle Iteration.
 - Einfache Anbindung an REST und **WebSocket** für die Multiplayer-Synchronisation.
- **Konsequenzen:** Schnelle UI-Entwicklung, gute Erweiterbarkeit (Dashboards, Live-Updates, Minispiele); kein Typsystem (bewusst, da Projektumfang überschaubar).

ADR-003 · Datenbank: PostgreSQL

- **Status:** ✓ Akzeptiert
- **Kontext:** Persistente Spielsitzungen, Flotten, Frachtaufträge, Hafendaten, Finanztransaktionen und synchronisierter Spielzustand über mehrere Spieler.
- **Entscheidung:** **PostgreSQL** als primäre relationale Datenbank.

- **Begründung:**
 - Starke Unterstützung relationaler Domänenmodelle (Spieler, Schiffe, Häfen, Verträge, Sessions).
 - Nahtlose Integration mit **Spring Data JPA / Hibernate**.
- **Konsequenzen:** Zuverlässige Persistenz; geringeres Risiko inkonsistenter Daten bei gleichzeitigen Spieleraktionen; einfaches Entity-Mapping per JPA-Repositories.

ADR-004 · Architektur-Einstieg: Monolith zuerst

- **Status:** ✓ Akzeptiert, später **abgelöst** durch ADR-005
- **Kontext:** Projektstart, hohe Unsicherheit über Domänenschnitt; Ziel war zunächst ein lauffähiger Prototyp.
- **Entscheidung:** Mit einem **monolithischen** Aufbau in einem **Monorepo** starten, statt sofort verteilt zu bauen.
- **Begründung:** Ein Monolith reduziert die Komplexität am Anfang erheblich; das verteilte System bleibt das langfristige Ziel.
- **Konsequenzen:** Schneller erster Prototyp; bewusste technische Schuld, die mit dem späteren Service-Schnitt (ADR-005) zurückgezahlt wurde.

ADR-005 · Schnitt in fünf Microservices + Gateway

- **Status:** ✓ Akzeptiert
- **Kontext:** Mit wachsender Domäne wurde der Monolith unübersichtlich; im Event Storming kristallisierten sich klare fachliche Cluster heraus.
- **Entscheidung:** Aufteilung in **session, player, freight, navigation, game-engine** plus **api-gateway** als einzigen Eingang.
- **Begründung:**
 - Klare fachliche Verantwortlichkeiten (Bounded Contexts), unabhängige Entwicklung und Deploybarkeit.
 - **Weitgehend azyklische Abhängigkeiten** (bewusstes Designziel); navigation ohne Abhängigkeiten, game-engine als Orchestrator oben. Einzige Ausnahme: die fire-and-forget-News-Meldung freight→session.
- **Konsequenzen:** Höhere Betriebskomplexität (mehr Prozesse, Netzwerk-Calls), dafür bessere Trennung und Skalierbarkeit. Erkauft wird das mit Inter-Service-Kommunikation und eventual-konsistenten Zuständen.

ADR-006 · Server-autoritatives Spielmodell mit Tick-Engine

- **Status:** ✓ Akzeptiert
- **Kontext:** Mehrere Clients müssen denselben Spielzustand sehen; Zufallsereignisse dürfen nicht manipulierbar sein.
- **Entscheidung:** **Sämtliche Logik serverseitig**. Ein zentraler Scheduler (@Scheduled, fixedDelay) treibt **Ticks** voran; Aktionen erhalten Start-/ End-Zeitpunkte, der Server kontrolliert den Ablauf.
- **Begründung:** Konsistenz und Kontrolle; „möglichst wenig Logik im Frontend“. Das Tick-Modell ersetzte einen früheren Ansatz mit vordefinierten Animationen.

- **Konsequenzen:** Robust gegen Client-Manipulation; Frontend wird „dumm“ und rein darstellend. Latenz-Effekte werden durch zeitbasierte Interpolation im Client kaschiert.

ADR-007 · Echtzeit-Kommunikation über WebSocket (STOMP)

- **Status:** ✓ Akzeptiert
- **Kontext:** Tick-Updates, Lobby-Änderungen und Ereignisse müssen ohne Polling bei allen Spielern ankommen.
- **Entscheidung:** **WebSocket mit STOMP**, getunnelt durch das Gateway; drei unabhängige Broker (session, freight, engine).
- **Begründung:** WebSockets sind bereits in Spring integriert und für Echtzeitkommunikation die einfachste Lösung. Reines Request/Response wäre für das Tick-Modell unpassend.
- **Konsequenzen:** Server-Push statt Polling; Topics pro Session (/topic/game/{id} etc.). Verbindungs-/Reconnect-Handling muss im Client bedacht werden.

ADR-008 · Synchrone Service-Kommunikation via RestTemplate

- **Status:** ✓ Akzeptiert
- **Kontext:** Services brauchen punktuell Daten voneinander (z. B. freight → player für die Reward-Buchung).
- **Entscheidung:** **Synchrones HTTP** mit RestTemplate für Service-zu-Service.
- **Begründung:** Einfach, ausreichend für die Last; kein zusätzlicher Message-Broker nötig. Echtzeit-Anteile laufen ohnehin über WebSocket.
- **Konsequenzen:** Geringer Infrastruktur-Overhead; dafür **zeitliche Kopplung** (ein langsamer Service bremst den Aufrufer). Für die Projektgröße ein bewusst akzeptierter Kompromiss.

ADR-009 · Database-per-Service in einem PostgreSQL-Container

- **Status:** ✓ Akzeptiert
- **Kontext:** Jeder Service soll seine Daten besitzen, ohne den Betriebsaufwand mehrerer DB-Server.
- **Entscheidung:** **Vier getrennte Datenbanken** (portsofcall_sessions, portsofcall_players, portsofcall_freight, portsofcall_navigation) in **einem** Postgres-Container, per Init-Skript angelegt; game-engine bleibt stateless.
- **Begründung:** Logische Trennung (kein Service liest fremde Tabellen) bei minimaler Infrastruktur. Mehrere DBs in einem Container sind für den Projektrahmen vertretbar.
- **Konsequenzen:** Keine Joins über Service-Grenzen; Referenzen nur per ID. Ein Container ist ein Single Point of Failure — für den Projektkontext akzeptabel.

ADR-010 · Keine gemeinsame „Common Library“ für DTOs

- **Status:** ✓ Akzeptiert
- **Kontext:** Es lag nahe, DTOs in eine geteilte Bibliothek auszulagern, damit alle Services dieselben Klassen nutzen.

- **Entscheidung:** **Bewusst keine** Common-Lib; jeder Service definiert seine DTOs selbst (ggf. dupliziert).
- **Begründung:** Ziel von DDD/Microservices sind **unabhängige** Services. Eine Common-Lib koppelt alle Services aneinander: Ändert sich die Lib, müssen alle Services angepasst und neu deployed werden → schlechtere Skalierung.
- **Konsequenzen:** Etwas Duplizierung bei DTOs, dafür echte Entkopplung und unabhängige Deploybarkeit.

ADR-011 · Domänenmodell nur dort, wo es Wert schafft

- **Status:** ✓ Akzeptiert
- **Kontext:** Frage, ob jeder Service ein reiches DDD-Domänenmodell braucht oder reine JPA-Entities reichen.
- **Entscheidung:** Reiches Modell in den **logiklastigen** Services (v. a. freight), **CRUD-nah** in navigation (statische Stammdaten).
- **Begründung:** Ein volles Domänenmodell für reine CRUD-Daten ist Over-Engineering; die fachliche Logik (Reise, Zoll, Schmuggel) soll dagegen sauber gekapselt sein.
- **Konsequenzen:** Angemessener Aufwand je Kontext; klare Trennung von Stammdaten und Verhalten.

ADR-012 · Schiffbesicherte Hypothek als Finanz-Hebel

- **Status:** ✓ Akzeptiert
- **Kontext:** Die Bank bot zunächst nur **unbesicherte Kredite** (Loan). Für mehr strategische Tiefe sollte ein Spieler Liquidität gegen den Einsatz eigener Vermögenswerte (Schiffe) gewinnen können — mit entsprechendem Risiko.
- **Entscheidung:** Einführung einer **Hypothek** (Mortgage) in service-player, besichert durch ein konkretes Schiff. Beleihungsgrenze **60 % des Schiffswerts**; bei anhaltendem Zahlungsausfall **pfändet** die Bank das Schiff (SEIZED) statt es nur als DEFAULTED zu markieren. Die Bewertung/Pfändung des Schiffs erfolgt über den ShipClient zu service-freight.
- **Begründung:** Wiederverwendung der bestehenden Kredit-Mechanik (Rating, Per-Tick-Tilgung, Verzugszins) bei minimaler Mehrkomplexität; die Besicherung schafft eine echte fachliche Differenzierung (Hebel vs. Totalverlust eines Schiffes). Die Finanzlogik bleibt im Eigentümer-Service des Kontostands (service-player), konsistent mit ADR-005/ADR-011.
- **Konsequenzen:** service-player erhält eine **gerichtete Abhängigkeit** zu service-freight (Schiffswert lesen, Schiff pfänden) — azyklisch, da freight nicht zurück auf player-Financen zugreift. Schiffe mit offener Hypothek sind nicht verkäuflich; der Pfand-Status muss in Flotte/UI sichtbar sein.

ADR-013 · API-Gateway als einziger Eingang (Spring Cloud Gateway)

- **Status:** ✓ Akzeptiert
- **Kontext:** Nach dem Service-Schnitt (ADR-005) hat das System fünf Backend-Prozesse auf den Ports 8081–8087. Der Browser braucht aber genau eine erreichbare Adresse;

außerdem hingen CORS-Regeln in der Monolith-Zeit als `@CrossOrigin` an jedem einzelnen Controller.

- **Entscheidung:** Ein Spring Cloud Gateway (reaktiv/WebFlux) auf Port 8080 ist der einzige öffentliche Eingang. Es routet per Pfad-Prädikat an die Services, setzt CORS global und tunnelt die WebSocket-Verbindungen (`ws://`-Schema im Routing-Eintrag) an die Upstreams.
- **Begründung:**
 - Routing, CORS und WS-Tunnel an einer Stelle statt verstreut über alle Services.
 - Minimale öffentliche Angriffsfläche; die internen Service-Ports bleiben privat.
 - Reaktives Gateway, weil WebSocket-Tunneling und globales CORS dort nativ unterstützt sind.
- **Konsequenzen:** Single Point of Entry (und damit -of-Failure) — für den Projektrahmen akzeptiert. Die Routing-Reihenfolge muss beachtet werden (spezifische Pfade vor Catch-alls); jeder neue Endpunkt erfordert einen Routing-Eintrag.

ADR-014 · Optimistic Locking gegen konkurrierende Auftragsannahme

- **Status:** ✓ Akzeptiert
- **Kontext:** Mehrere Spieler sehen denselben Frachtmarkt in Echtzeit. Zwei könnten denselben offenen Auftrag im selben Moment annehmen — ein klassischer Lost-Update- bzw. Doppelvergabe-Fall.
- **Entscheidung:** `CargoContract` (und `GameSession`) tragen ein `@Version`-Feld. Beim gleichzeitigen Schreiben gewinnt der erste Commit; der zweite läuft in eine `OptimisticLockException`. Ein `GlobalExceptionHandler` je Service übersetzt diese in eine definierte Fehlerantwort an den Client.
- **Begründung:**
 - Optimistic Locking passt zu einer Domäne mit seltenen echten Kollisionen besser als pessimistische DB-Sperren — keine Lock-Verwaltung, keine Deadlock-Gefahr, kein Durchsatzverlust im Normalfall.
 - Bei einem Konflikt ist das fachlich korrekte Ergebnis („nur einer bekommt den Auftrag“) automatisch erfüllt.
- **Konsequenzen:** Der Client muss den Konfliktfall behandeln (Auftrag bereits vergeben) statt ihn zu ignorieren. Minimaler Mehraufwand (eine Spalte, ein Handler) gegen garantierte Korrektheit bei Nebenläufigkeit.

ADR-015 · Distanzbasierte Schiffsgeschwindigkeit & Spritverbrauch pro Klasse

- **Status:** ✓ Akzeptiert (löst das frühere „ein Wegpunkt pro Tick“-Modell ab)
- **Kontext:** Schiffe fahren entlang handgezeichneter Routen (Polylinien aus dem `MapEditor`). Ursprünglich rückte ein Schiff um genau einen Wegpunkt pro Tick vor, und Sprit wurde pro Tick verbraucht. Das koppelte die Reisedauer an die Punktdichte der Route (statt an ihre echte Länge) und die Reichweite an die Geschwindigkeit — kleine, schnelle Schiffe hatten dadurch paradoxerweise die größte Reichweite.
- **Entscheidung:** Beide Werte werden distanzbasiert im Koordinatenraum (0..1000) modelliert. $\text{Speed} = \text{Distanz-Einheiten pro Tick}$ ($\text{Reisezeit} = \text{Streckenlänge} / \text{Speed}$); $\text{Verbrauch} = \text{Distanz} \times \text{Sprit-pro-100}$ ($\text{Reichweite} \approx 10000 / \text{Verbrauch}$, unabhängig von der Geschwindigkeit). Das Schiff trackt `routeProgress`; Lotse, Midpoint-Minispiel und Ankunft werden über Distanzschwellen geprüft. Die Klassenwerte liegen zentral in `ShipClassConfig` und sind in `application.properties` tunebar.
- **Begründung:**

- „Wie weit komme ich mit einer Tankfüllung" und „wie lange dauert die Fahrt" sind physikalisch Funktionen der Strecke; das entkoppelt Tempo und Reichweite sauber von der Zeichendichte und voneinander.
- Es ergeben sich klare Klassen-Trade-offs: High-Tech = viel Volumen + Langstrecke, dafür langsam; Low-Cost = klein + schnell, dafür kurze Reichweite (muss zwischentanken).
- **Konsequenzen:** Neues Feld `routeProgress`; das alte `fuelConsumptionPerTick` ist vestigial. Frontend und Backend teilen `routeProgress/distancePerTick` im DTO (eine Schemaänderung erfordert beidseitigen Deploy); das Frontend interpoliert auf der Kurve via `routeProgress + tickFraction × distancePerTick`. `service-freight` muss nach Property-Änderungen neu starten.

ADR-016 · Containerisierung & Deployment via Docker Compose

- **Status:** ✓ Akzeptiert
- **Kontext:** Nach dem Service-Schnitt (ADR-005) besteht das System aus fünf Backend-Prozessen, einem API-Gateway, einer Datenbank und dem Frontend. Für Entwicklung, Präsentation und reproduzierbares Hochfahren braucht es einen Weg, all diese Teile mit korrekter Start-Reihenfolge und einheitlicher Konfiguration in einem Schritt zu starten — ohne dass jeder im Team Ports, DB-Namen und Service-URLs manuell zusammensteckt.
- **Entscheidung:** Das gesamte System wird über eine einzige `docker-compose.yml` orchestriert. Jeder Backend-Service wird aus demselben `./backend-Build-Context` über ein `SERVICE_NAME`-Build-Argument gebaut (ein Dockerfile, mehrere Images). Eine einzelne PostgreSQL-Instanz (`postgres:15`) hält alle vier Datenbanken, angelegt durch das Init-Skript `./docker/postgres-init/01-create-databases.sql` (vgl. ADR-009). Das Frontend wird als statischer Build über Nginx ausgeliefert. Öffentlich erreichbar sind nur das `api-gateway` (8080) und das Frontend (5173); alle Backend-Services kommunizieren ausschließlich über das interne Docker-Netz; alle anderen Services kommunizieren ausschließlich über das interne Docker-Netz.
- **Begründung:**
 - Ein-Kommando-Start (`docker compose up`) statt manuellem Hochziehen von sieben Prozessen — entscheidend für Demo- und Präsentationsreife.
 - Healthcheck-gesteuerte Start-Reihenfolge (`depends_on` mit `condition: service_healthy/service_started`): Postgres muss bereit sein, bevor Services starten; `navigation` → `player` → `freight` → `session` → `game-engine` → `gateway` respektiert die fachlichen Abhängigkeiten.
 - Einheitliche Konfiguration per Umgebungsvariablen (`DB_HOST`, `SERVICES_*_URL`), sodass dieselben Images lokal und im Compose-Netz laufen.
 - Statisches Nginx-Frontend statt Vite-Dev-Server im Produktionsbild: kleineres Image, kein Dev-Toolchain-Ballast im Deployment.
- **Konsequenzen:** Reproduzierbares Hochfahren des Gesamtsystems mit minimaler Infrastruktur. Persistenz über ein benanntes Volume (`pgdata`); `restart: unless-stopped` fängt transiente Abstürze ab. Der eine Postgres-Container bleibt ein Single Point of Failure (bewusst akzeptiert, vgl. ADR-009). Jeder neue Service erfordert einen Compose-Eintrag inkl. Abhängigkeits- und URL-Konfiguration.

ADR-017 · Keine Authentifizierung/Autorisierung im MVP

- **Status:** ✓ Akzeptiert (bewusste Scope-Entscheidung)
- **Kontext:** Das Spiel läuft als Multiplayer-System über das api-gateway als einzigen Eingang (ADR-013). Spieler- und Session-Identität werden über IDs (playerId, sessionId) in den Requests geführt. Es stellte sich die Frage, ob für das MVP eine echte Benutzer-Authentifizierung (Login, Token, geschützte Endpunkte) eingeführt werden soll.
- **Entscheidung:** Für das MVP wird bewusst keine Authentifizierung oder Autorisierung implementiert. Es gibt kein Spring Security, keine Login-/Token-Mechanik und keine geschützten Endpunkte; die Spieleridentität wird allein über die in den Requests mitgeführten IDs bestimmt.
- **Begründung:**
 - Der Fokus des MVP liegt auf der Spielmechanik (Frachtmarkt, Schiffe, Häfen, Ereignisse, Bank) und der Multiplayer-Synchronisation, nicht auf Identitätsmanagement.
 - Das Spiel wird in einer vertrauenswürdigen Umgebung (lokale Demo, Präsentation, kein öffentlicher Betrieb) genutzt; ein vollwertiger Auth-Layer wäre für diesen Rahmen Over-Engineering.
 - Das Gateway hält die internen Service-Ports ohnehin privat (ADR-013), was die Angriffsfläche im Demo-Kontext begrenzt.
- **Konsequenzen:** Requests sind nicht gegen gefälschte oder fremde playerId/sessionId geschützt — ein Client könnte prinzipiell im Namen eines anderen Spielers handeln. Für den öffentlichen Betrieb müsste nachträglich ein Auth-Layer (z. B. am Gateway, mit Token-Validierung und serverseitiger Identitätsbindung) ergänzt werden; das ist als bewusste technische Schuld dokumentiert und kein Versehen.

4. Nicht-funktionale Anforderungen

Die funktionalen Anforderungen (Abschnitt 1) beschreiben, **was** das System tut; die folgenden Qualitätsanforderungen beschreiben, **wie gut** es das tun muss. Sie waren die eigentlichen Treiber der Architektur — fast jede Entscheidung in Abschnitt 2 und jeder ADR in Abschnitt 3.2 lässt sich auf eine dieser Qualitäten zurückführen.

4.1 Qualitätsziele (priorisiert)

#	Qualität	Konkretes Ziel (Szenario)	Priorität
Q1	Konsistenz / Integrität	Alle Spieler einer Session sehen zu jedem Tick denselben Zustand; kein Client kann den Spielverlauf manipulieren.	Muss
Q2	Echtzeit / Performance	Ein Tick wird unterhalb des Tick-Intervalls (5 s) verarbeitet und an alle Clients gepusht; die Schiffsanimation bleibt flüssig (60 FPS).	Muss
Q3	Skalierbarkeit	2–6 Spieler pro Session und mehrere parallele Sessions ohne gegenseitige Beeinflussung.	Muss
Q4	Robustheit / Verfügbarkeit	Ein Service-Absturz oder ein fehlerhafter Tick führt nicht zu inkonsistentem Zustand; Services erholen sich ohne manuellen Eingriff.	Soll
Q5	Wartbarkeit / Modifizierbarkeit	Balance-Parameter sind ohne Codeänderung anpassbar; ein Service ist unabhängig von	Soll

		den anderen deploybar.	
Q6	Betreibbarkeit / Portabilität	Das Gesamtsystem (DB + 5 Services + Gateway + Frontend) startet mit einem einzigen docker compose up.	Soll

4.2 Umsetzung je Qualitätsziel

Q1 — Konsistenz & Integrität (server-authoritative)

- Sämtlicher Spielzustand, alle Berechnungen und alle Zufallswürfe liegen serverseitig; der Client ist rein darstellend (ADR-006). Damit sind clientseitige Manipulationen (Geld, Position, Würfelergebnisse) wirkungslos.
- Konkurrierende Auftragsannahme wird durch Optimistic Locking (@Version auf CargoContract) deterministisch aufgelöst: Der zweite Schreibvorgang läuft in eine OptimisticLockException, die ein GlobalExceptionHandler je Service in eine saubere Fehlerantwort übersetzt (ADR-014).
- Der GameState wird pro Tick zuerst vollständig gebaut und erst dann gebroadcastet — schlägt der Aufbau fehl, laufen Tick-Zähler und Schiff-Snapshot nicht auseinander.
- Keine Fremdschlüssel über Servicegrenzen; jeder Service ist alleiniger Eigentümer seiner Daten (ADR-009) — keine geteilten, divergierenden Kopien.

Q2 — Echtzeit & Performance

- Server-Push statt Polling: Tick und GameState gehen über WebSocket/STOMP an /topic/game/{id} (ADR-007). Kein Client fragt aktiv ab.
- Der Scheduler nutzt @Scheduled(fixedDelay) statt fixedRate — ein neuer Tick startet erst N ms *nach* Ende des vorigen, sodass sich Ticks bei einer kurzen Lastspitze nicht stauen.
- Flüssige Darstellung trotz 5-s-Takt: Das Frontend interpoliert die routeProgress-Werte in einer 60-FPS-RAF-Schleife zwischen zwei Broadcasts (eased toward target, ohne Extrapolation) und misst das Tick-Intervall dynamisch — die Animation bricht nicht, wenn die Tick-Rate zum Testen verändert wird.

Q3 — Skalierbarkeit

- Microservice-Schnitt mit Database-per-Service und stateless game-engine (ADR-005, ADR-009): logiklastige Teile (freight) sind unabhängig von den Stammdaten (navigation) skalier- und deploybar.
- Alle veränderlichen Daten sind session-gescoped (sessionId als Pflichtfeld) — mehrere Sessions laufen parallel, ohne dass Spieler fremde Verträge oder Schiffe sehen.

Q4 — Robustheit & Verfügbarkeit

- docker compose startet die Services mit restart: unless-stopped und depends_on ... condition: service_healthy gegen einen Postgres-Healthcheck — ein abgestürzter Service kommt von selbst wieder, und nichts startet, bevor die DB bereit ist.

- Jeder Service besitzt einen `GlobalExceptionHandler`, der Fehler (u. a. Optimistic-Lock-Konflikte) in definierte HTTP-Antworten statt in 500er-Leaks übersetzt.
- „GameState zuerst bauen, dann broadcasten“ (siehe Q1) ist zugleich die Fehlertoleranz im Tick: lieber ein Tick übersprungen als ein inkonsistenter Broadcast.

Q5 — Wartbarkeit & Modifizierbarkeit

- Sämtliche Balance-Parameter liegen als Konfiguration vor (Tabelle in Abschnitt 2.2), nicht hartcodiert — Tick-Intervall, Zoll-Wahrscheinlichkeiten, Kredit-/Hypothekenzinsen sowie Schiffsgeschwindigkeit und -verbrauch sind per Property + Neustart änderbar (ADR-015).
- Klare Bounded Contexts ohne gemeinsame DTO-Bibliothek (ADR-010): eine Änderung in einem Service zwingt nicht alle anderen zum Re-Deploy.
- Jede wesentliche Entscheidung ist als ADR dokumentiert (Abschnitt 3.2) — das „Warum“ bleibt nachvollziehbar.

Q6 — Betreibbarkeit & Portabilität

- Ein `docker compose up` startet vier PostgreSQL-Datenbanken (ein Container, Init-Skript), alle fünf Services, das Gateway und das Frontend.
- Service-Adressen sind über Umgebungsvariablen überschreibbar (lokal `localhost`, im Compose-Netz die internen Hostnamen) — derselbe Build läuft lokal und im Container.

4.3 Bewusste Nicht-Ziele

Folgende Qualitäten wurden für den Projektrahmen explizit zurückgestellt:

- **Authentifizierung / Autorisierung:** Es gibt kein Login und keinen Auth-Provider (Kontextdiagramm, Abschnitt 2). Spieler werden nur über einen Session-Code unterschieden; eine Benutzerverwaltung war nicht Teil des MVP.
- **Persistenz über Disconnect (Save/Resume):** Bewusst offen (Abschnitt 1.3) — der Fokus lag auf Stabilität und Präsentationsreife einer laufenden Runde.
- **Hochverfügbarkeit der Datenbank:** Alle vier DBs liegen in einem Postgres-Container — ein bewusst akzeptierter Single Point of Failure für den Projektrahmen (ADR-009).
- **Horizontale Replikation der game-engine:** Sie läuft als einzelne Instanz mit einem zentralen Scheduler; mehrere Instanzen würden doppelte Ticks erzeugen und sind nicht vorgesehen.

5. Glossar

Begriff	Bedeutung
Tick	Kleinste Zeiteinheit der Simulation; <i>1 Tick = 1 Spieltag</i> . Vom Scheduler alle 5 s ausgelöst.
Session	Eine Spielrunde mit bis zu 6 Spielern, gemeinsamer Welt und synchronem Zeitmodell.
Reederei	Das Unternehmen eines Spielers (im Code: Player mit Kontostand und Flotte).
Flotte	Alle PlayerShip-Objekte eines Spielers.
Frachtauftrag	CargoContract: Herkunfts-/Zielhafen, Reward, Kapazität, Frist; legal oder illegal (Schmuggel).
Regress	Vertragsstrafe bei nicht (rechtzeitig) erfülltem Auftrag (Forfeit 50 % + 80/Tick Verspätung).
Wegpunkt (Waypoint)	Punkt einer Route-Polylinie. Das Schiff bewegt sich distanzbasiert entlang der Strecke (routeProgress += distancePerTick)
Route / Alternativroute	Vorberechneter Pfad zwischen zwei Häfen; alternativeWaypoints bei politischer Sperre.
Lotse (Pilot)	Kostenpflichtiger Service (500 Taler) fürs An-/Ablegen; Alternative: manuelles Anlegen (Minispiel).
Zollkontrolle (Customs)	Zufällige Kontrolle (35 %) im Zielhafen; bei entdecktem Schmuggel Strafe + Festsetzung.
Bestechung (Bribe)	Versuch, die Zollkontrolle abzuwenden; bei Misserfolg ist die Zahlung verloren und der Schmuggel wird sicher entdeckt (Konfiszierung + Bußgeld).
Schmuggel	Illegale Zusatzfracht mit höherem Ertrag, aber Entdeckungsrisiko.
Kredit (Loan)	Unbesicherter Bankkredit; per-Tick getilgt, Status ACTIVE → OVERDUE → DEFAULTED.
Hypothek (Mortgage)	Durch ein Schiff besicherter Kredit (max. 60 % Schiffswert); bei Ausfall pfändet die Bank das Schiff (SEIZED).
Bonität / Rating	Aus Cash und Schulden errechnetes Kredit-Rating (S+ bis E), das den Zinssatz bestimmt.
Politisches Ereignis	Globale Routen-/Hafen-Sperre (Suez, Panama, Nordatlantik, Hormuz).
Minispiel	Client-seitige Interaktion als Reaktion auf ein Server-Event (Ratten, Mann über Bord, Felsen ausweichen, Anlegen).
Tick-Engine	service-game-engine: stateless Orchestrator, der pro Tick alle Services koordiniert und broadcastet.
Bounded Context STOMP	Fachlich abgegrenzter Bereich; entspricht hier je einem Microservice. Text-Protokoll über WebSocket für die Topic-basierte Echtzeitkommunikation.
ADR	Architecture Decision Record — dokumentierte Entscheidung mit Kontext, Begründung und Konsequenzen.

DoR / DoD	Definition of Ready / Definition of Done — Eintritts-/Abschlusskriterien einer User Story.
Leaderboard	Rangliste nach Reinvermögen (Cash – Schulden); der Flottenwert geht bewusst nicht ein. Wird über den Endpoint bei Bedarf berechnet.

Stand: 30. Juni 2026.